FP 6 (Haskell 2): Basic types & functions

Cael Marquard mrqcae001@myuct.ac.za
Department of Computer Science
University of Cape Town
(slides adapted from Dr Sabre Didi)



Aim of this lecture

At the end of this lecture you should:

- Know about basic types in Haskell
- Understand functions with multiple arguments, and function currying
- Be able to define your own functions
- Know what a tuple is and how to use it
- Know what product and sum types are, and the differences between them
- Be able to define your own algebraic datatypes

What is a type?

- **A type is a name for a collection of related values.**
- For example, in Haskell the basic type `Bool` contains the values `True` and `False`.
- A type lets you restrict a binding (e.g. variable, function argument, function returns) to contain **only certain kinds of things**.
- The function `isEven :: Int -> Bool` accepts an Int and returns a Bool
  - Int: “A fixed-precision integer type with at least the range [-2^29 .. 2^29-1].”
  - Can this function accept the input `2.5`? What about `“dog”`?
  - What can you tell me about the function `myDiv :: Int -> Int -> Float`?

Types in Haskell

If evaluating an expression `e` would produce a value of type `T`, then `e` has type `T`, written as

```
e :: T
```

Every well-formed expression has a type. Moreover, this can often be **inferred at compile-time based on context.**

```
x :: Int = 1

y = x  -- What is the type of y?
```

Typechecking

All type errors are found at compile time, which makes programs safer and faster by removing the possibility of runtime type errors.

The following code **will not compile** in Haskell:

```
addNumbers :: Int -> Int -> Int
addNumbers x y = x + y
addNumbers "Hello" 1
```

If something like this were buried deep in a *Python* codebase—say, we try to `add_numbers(customerBalance, transaction)`, not realising that `transaction` refers to the transaction *ID* and not amount—the software could chug along fine for a while… Until that code runs and blows the whole thing up in your face.

Basic types

Haskell has some basic types to be aware of:

- **()** - the **unit type**. This means “no value” and is useful for functions that do IO but don’t return anything (e.g.: the `print` function)
- **Bool** - True or False
- **Char** - single characters - one unicode codepoint
  - Maybe don’t read these if you don’t want to confuse yourself, to be honest. [AR: “Let’s stop ascribing meaning to code-points” by Manish Goregaokar] [AR: “Text Rendering Hates You” by @Gankra]
- **String** - a string of characters
- **Int** - fixed-size integers
- **Integer** - arbitrary-size integers (e.g. BigNum / BigInt)
- **Float / Double** - floating point numbers
- **List** - a list of values of the same type

Recap quiz (3 points each)

1. What does the `map` function (from the prelude) do in Haskell?
   - **a.** It relates to GPS (specifically, plotting waypoints on a **map**)
   - **b.** It applies a function to every element of a list, **mapping** it from one thing to another
   - **c.** It returns the index of each element in a list (**maps** them out)
   - **d.** It is used to lift/**map** a pure value into the structure of the `IO` monad
2. What does a function being **“pure”** mean (in FP)?
   - **a.** It means that the function is written **purely** in the Haskell language (it doesn’t use any C/system libraries via FFI)
   - **b.** It is another word for a total function: it must terminate/halt and return a value (it is **pure** from potential diverging code)
   - **c.** The function is defined **purely** with standard-library functions (no user functions)
   - **d.** It means the function has no side-effects—it **purely** computes the return value

Lists

- A list is an **ordered sequence** of values, all of the **same type**
- **Length is unrestricted** (like list in Python, ArrayList in Java)
- We can write a list of `Int` as `[Int]`
- **Type is unrestricted**: we can have a list of **anything** (even lists or functions)
  - **BUT** every element of a list must be the **same type**

`[False, True, False] :: [Bool]` ✅

`[1, 2, 3, 4, 5] :: [Int]` ✅

`[1.0, 1.2, 1.4, 5.0] :: [Float]` ✅

`[[1], [1, 2], [1, 2, 3]] :: [[Int]]` ✅

`[] :: [Int]` ✅

`[1.0, False]` ❌ (must all be the same type!)

Lists: cons

Lists are represented similar to linked-lists in other languages. An element of the type List is either:

1. The empty list, []

OR

2. The current element joined with the rest of the list. This ‘join’ is done through an operator called “cons”, and is written as : for shorthand

```
-- The | here literally means or: a value of List can either be the LHS ([])
-- or the RHS (a : List a)
data List a = [] | a : List a

-- so, these lines are all equivalent
[1, 2, 3, 4]
1:2:3:4:[]
1:(2:(3:(4:[])))
```

Tuples

- A tuple is a **fixed-length,** ordered sequence of values**, possibly of different types**.
- They can be used to group values without defining a whole new type (e.g. to return multiple values from one function—e.g., a person's name (String) and age (Int)).
- We write a tuple as `(Type1, Type2, Type3, etc)`

```
(False, True, False) :: (Bool, Bool Bool) ✅
```

```
(False, 1, "Hello") :: (Bool, Int, String) ✅
```

- A pair is a common type of tuple, which consists of two elements. The functions `fst` and `snd` are defined on pairs.

```
(3, False) :: (Int, Bool).
```

```
snd (3, False) = False
```

```
fst (3, False) = 3
```

Functions

A function maps values of one type to another.

```
not :: Bool -> Bool
```

```
even :: Int -> Bool
```

The type of a function mapping T1 to T2 is written as

```
T1 -> T2
```

Taking multiple arguments

Functions can take multiple arguments, as in other languages.

```
addVals :: Int -> Int -> Int

addVals a b = a + b
```

But wait - **why did we write Int -> Int -> Int** and not something like Int, Int -> Int?

Let us approach this from the opposite way around.

Function currying: rationale

Suppose we have a list of numbers, and we want to double each element. We could define a function to do this, and map it over the list:

```
double :: Int -> Int
double x = x * 2
map double list
```

But, we could **also** use a **partial application** of the function `* :: Int -> Int -> Int`

```
map (2 *) list
```

The expression `(2 *)` **calls the * function with one argument**. Thus, this expression now has type `Int -> Int`. You can think of it like giving the function one of its two arguments, and letting it 'wait' for the second one, which is given by `map`.

Function currying

Trivia: this concept is named after Haskell Curry, an American mathematician and computer scientist

Let us return to addVals

```
addVals :: Int -> Int -> Int
addVals a b = a + bc
```

The true reason that we write addVals's type as Int -> Int -> Int is because it is, under the hood, **not a function of two arguments!** It a function that **takes an Int, and returns a function that maps an Int to an Int**. It is like this to give us the flexibility to curry our functions.

```
addVals :: Int -> (Int -> Int)
```

Function currying: syntax

As we have already seen, these two syntaxes are equivalent

```
addVals :: Int -> Int -> Int
addVals 1 2

addVals :: Int -> (Int -> Int)
((addVals 1) 2)
```

This is because the -> arrow is **right-associative** and function application is **left-associative**.

Defining functions: conditionals & guards

We can define functions with if/else clauses as in other languages:

```
abs :: Int -> Int
abs n = if n >= 0 then n else -n

signum :: Int -> Int
signum n = if n < 0 then -1
           else if n == 0 then 0
           else 1
```

We can also use **guarded equations,** which are similar to piecewise functions in mathematics, or switch-case / match statements in other languages

$$f(x) = \begin{cases} -1 & x < -1 \\ x & -1 \leq x \leq 1 \\ 1 & x > 1 \end{cases}$$

```
f :: Float -> Float
f x | x < 0 = -1
    | x > 1 = 1
    | otherwise = x
```

Defining functions: pattern matching

We can also define functions by **pattern matching on their arguments:**

```
isPet :: String -> Bool
isPet "dog" = True
isPet "cat" = True
-- just give a variable name (or _, if the value isn't used) to match any value
-- if the function has multiple variables, each name must be unique (except for _)
isPet var = False
```

Patterns are matched **in definition order**. So, even though the value "dog" would match the `isPet var = False` definition, the `isPet "dog" = True` definition takes precedence, and the function returns True.

Pattern matching on lists

- We spoke about how lists are, internally, 1) the empty list OR 2) an element + the rest of the list. The cons operator (:) is used to join an element to the front of a list.
- **We can also use cons notation in pattern matching** (remember to wrap in () due to operator precedence)
- Here, head returns the first element of a list, and tail everything *except* that.
- Note that the patterns for an empty list [] are undefined for both. GHC will warn you that this is, therefore, a *partial* function. We will return to this later…

```
data List a = [] | a : List a
```

```
head :: [Int] -> Int
head (x:_) = x

tail :: [Int] -> [Int]
tail (_:xs) = xs
```

Quiz

1. (2 points) What is a tuple? (Give the most correct answer)
   - **a.** Variable length collection of homogenous types
   - **b.** Variable length collection of heterogeneous types
   - **c.** Fixed length collection of homogenous types
   - **d.** Fixed length collection of possibly heterogeneous types
2. (2 points) What is the definition of a list?
   - a. `data List a = [] | a : []`
   - b. `data List a = a | (a, [])`
   - c. `data List a = [] | a : List a`
   - d. `data List a = [] | a, List a`

Defining types

As in other languages, you can define your own types in Haskell. We will discuss defining your own **algebraic data types** and also how to make them **polymorphic**, in this course.

> **Algebraic datatype (definition): a data type made up of other data types; a composite data type. Algebraic datatypes include sum types and product types.**

This is quite similar to the concept of a class from Python (though there isn't inheritance in Haskell's algebraic datatypes) or a struct from C/C++/Rust.

Defining types: the algebra & sum types

Algebraic data types are so named because they are constructed using an algebra with the operators *sum* and *product*. **To contextualise this, think about a type as a set of possible values.**

A **sum type** is a type where its values can be one of many things (i.e., you are **adding** each thing to the set of possible values for that type). E.g., if our sets are A = {1, 2} and B = {foo, bar}, A + B = {1, 2, foo, bar}

```
-- Values of Suit can only be hearts, clubs, spades, or diamonds.
data Suit = Hearts | Clubs | Spades | Diamonds
x :: Suit = Hearts -- this is how we instantiate a value of Suit

-- Values of IntOrStr can be anything from Int or String
data IntOrStr = AnInteger Int | AString String
y :: IntOrStr = AnInteger 1 -- this is how we instantiate a value of IntOrStr
z :: IntOrStr = AString "Hello there"
```

Defining product types

A **product type** is a type where its **values must have *each of one of many things*** (i.e., you are creating the **cartesian product** of the input sets). E.g, if our sets are A = {1, 2} and B = {foo, bar}, the cartesian product A x B = {(1, foo), (2, foo), (1, bar), (2, bar)}

```
-- Here the Int and Str are unnamed members
-- They can be extracted via pattern matching
data IntAndStr = IntAndStr Int String
x :: IntAndStr = IntAndStr 1 "hello there"
```

Defining product types: named members with record syntax

You can give the elements of a product types names using *record syntax:*

```haskell
-- Here, number and street are named members
data Address = Address
    { number :: Int
    , street :: String
    } -- yes, this is the indentation convention for Haskell

-- We can create a value like this OR in the normal way
addr = Address { number = 1, street = "a" }
addr = Address 1 "a"

-- We can get the number like this
number addr

-- We can return a new Address with the number changed like this
-- (remember, Haskell values are immutable, so it's always a copy)
addr { number = 2 }
```

Exercise

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

- Write your own Person datatype, comprising of an integer age and string name.
- Then, create a list of people and filter it to people >= 18 years old (using the `filter` function + your own `isAdult` function)
- Try writing the `isAdult` function in three different ways: as a boolean expression, with a guarded expression, and with an if/else expression.
- (Extension) Try writing a *recursive function* to filter the list, rather than using the built-in `filter` function. Here you could use list pattern matching and cons-notation.

Assigned reading & assistance

As revision for today:

- (LYAH) Starting Out: Baby's first functions
- (LYAH) Starting Out: Texas ranges
- (LYAH) Higher Order Functions: Curried functions

As preparation for next lecture:

- (LYAH) Starting Out: I'm a list comprehension
- (LYAH) Starting Out: An intro to lists

If you have any queries, email them to me at mrqcae001@myuct.ac.za. I am happy to do office hours by appointment (also by email)